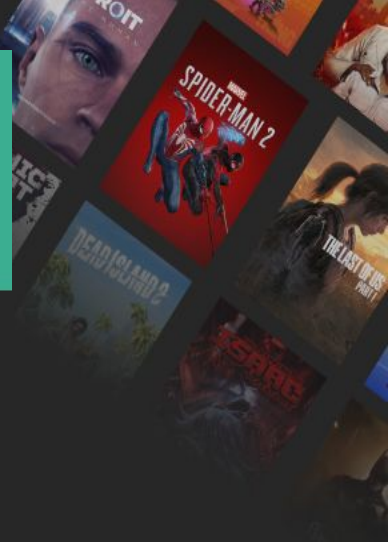
Gamelab



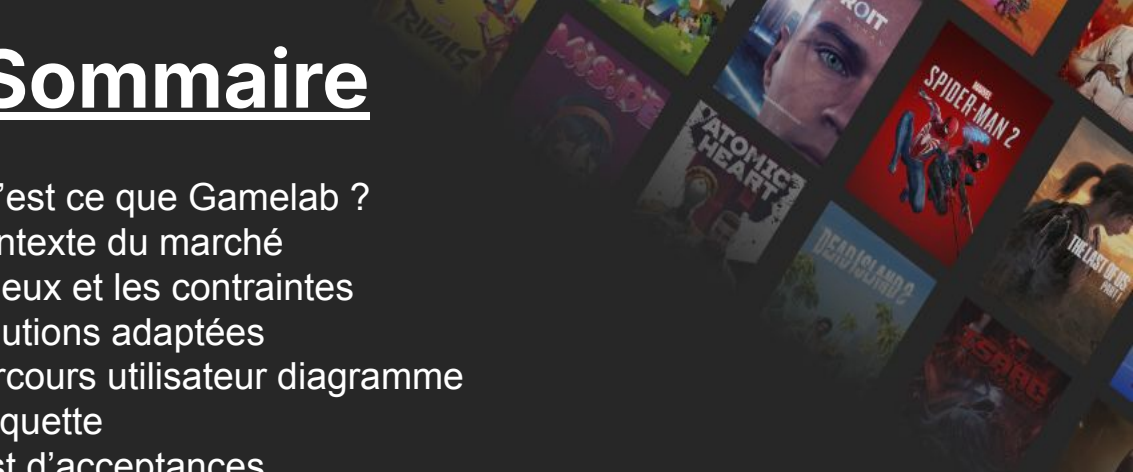
Find, Rate, Collect ….

Matthieu Nowak

ForEach ACADEMY

almadia

Sommaire



1. Qu’est ce que Gamelab ?
2. Contexte du marché
3. Enjeux et les contraintes
4. Solutions adaptées
5. Parcours utilisateur diagramme
6. Maquette
7. Test d’acceptances
8. Schéma MCD
9. Schéma d’architecture
10. Organisation
11. Les API
12. Docker de l’api Gamelab
13. Landing Page
14. L’application
15. Veille Technologique
16. Améliorations futures
17. Conclusion

1.Qu’est ce que Gamelab ?

Mobile / Web



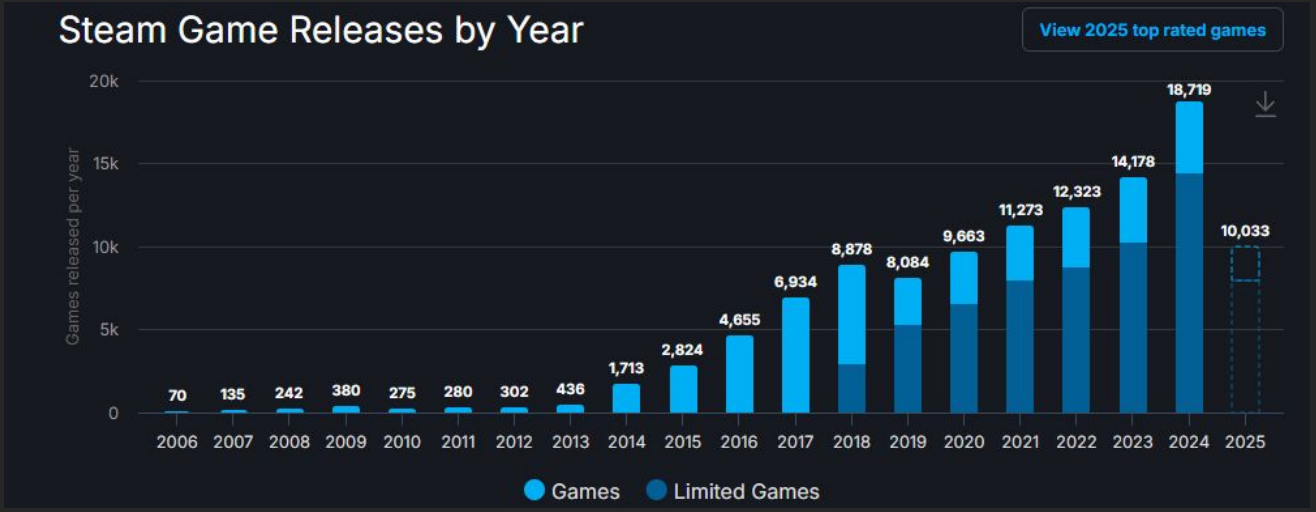
- 🔍 Find
- ❤️ Rate
- 🗂️ Collect
- 🎮 +300 000 Games

1.Qu’est ce que Gamelab ?



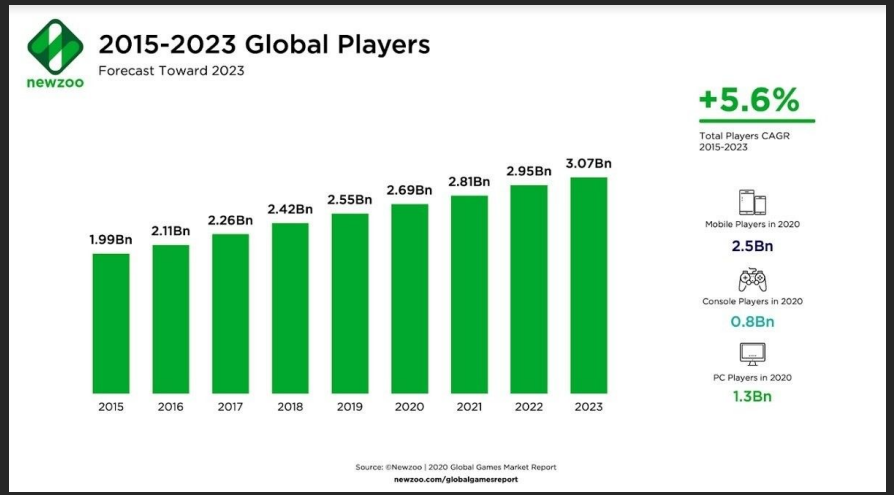
2.Contexte du marché

- 2023 → 14 500 nouveaux titres (40/60 JPJ)
- 2023 → 2025 → + 4 Mds Joueurs monde
- 2024 → 19 Mds € sur Steam pour jeux non joués



2.Contexte du marché

- 2023 → 14 500 nouveaux titres (40/60 JPJ)
- 2023 → 2025 → + 3 Mds Joueurs monde
- 2024 → 19 Mds € sur Steam pour jeux non joués



3.Enjeux et les contraintes



🎯 Enjeux

- Saturation du marché
- Adaptation Multiplateforme
- Croissance des utilisateurs

Contraintes

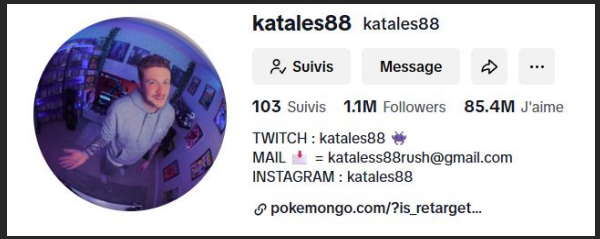
- Absence sur iOS
- Ressources financières et techniques

4.Solutions



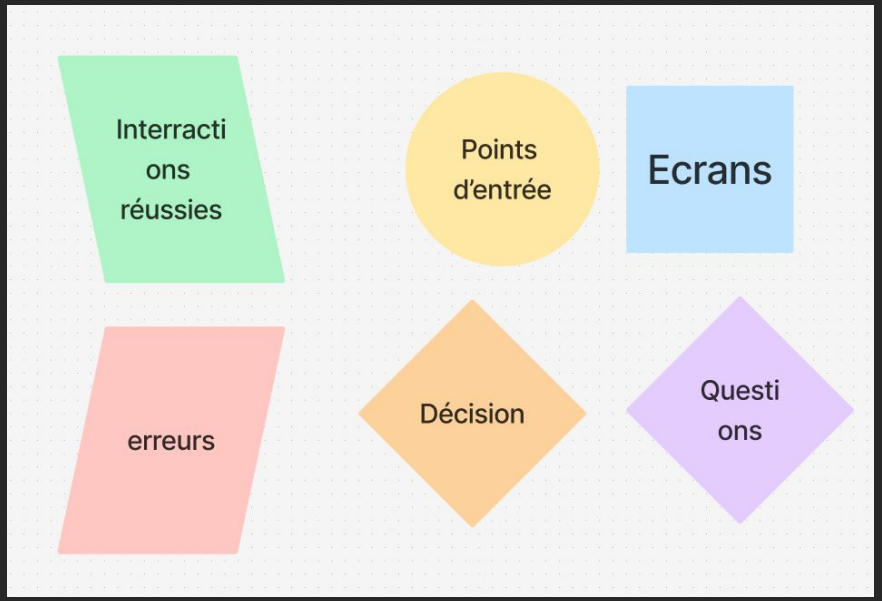
- Financement participatif (crowfunding)
- Partenariats stratégiques (Influenceurs, Acquisition)
- Publicité et promotions
- Achats intégrés

tipeee.com



5.Parcours utilisateur diagramme

- https://www.figma.com/board/Yq9C8v963lUbBty8hxMJVf/Diagramme-User-flow---Matthieu-NOWAK-%3A-Gamelab?node-id=0-1&t=fIfpfLkWpZPUzNM3-1



6.Maquette

https://www.figma.com/design/fc410BBFqEVlxb3AhRGxex/GAMELAB?node-id=0-1&p=f&t=nNKrBDotFBZ0OeLp-0



7.Test d’acceptances



User Story : Recherche de jeu et ajout à une collection personnalisé

Titre :

En tant qu’utilisateur, je veux pouvoir rechercher un jeu et l’ajouter à ma collection pour organiser mes jeux préférés.

Description :

En tant qu’utilisateur de Gamelab, je souhaite rechercher un jeu dans la base de données via la barre de recherche afin de pouvoir l’ajouter facilement à mes collections organisées dans des dossiers.

Critères d’acceptation :

- L’utilisateur peut entrer le nom du jeu dans la barre de recherche.
  - Une liste de résultats pertinents s’affiche en temps réel pendant la saisie.
  - L’utilisateur peut sélectionner un jeu parmi les résultats affichés.
  - Après sélection, l’utilisateur a l’option d’ajouter le jeu à une collection existante ou d’en créer une nouvelle.
  - Le jeu est visible dans la collection après ajout.
  - Un message de confirmation s’affiche une fois le jeu ajouté avec succès.

Tests d'acceptation

Scénario 1 : Recherche de jeu réussie et ajout à la collection

- **Étant donné** qu'un utilisateur est connecté à Gamelab,
- **Quand** il tape "Zelda" dans la barre de recherche,
- **Alors** une liste de résultats correspondant aux jeux "Zelda" s'affiche.
- **Quand** il sélectionne "The Legend of Zelda: Breath of the Wild" dans les résultats,
- **Et** qu'il choisit l'option "Ajouter à la collection",
- **Et** qu'il choisit un dossier existant ou créer un nouveau dossier nommé "Zelda",
- **Alors** le jeu est ajouté à la collection "Zelda" ou au dossier existant,
- **Et** un message de confirmation s’affiche : "The Legend of Zelda: Breath of the Wild ajouté avec succès à Zelda."

Scénario 2 : Aucune correspondance trouvée

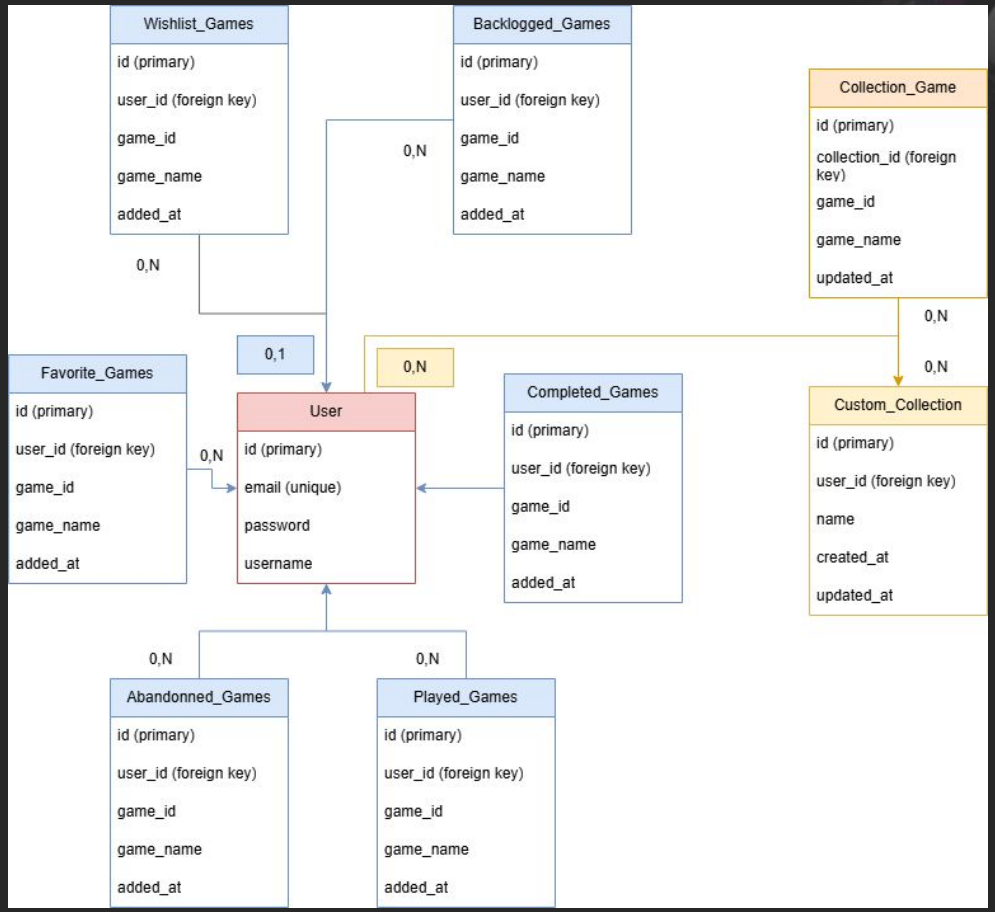
- **Étant donné** qu'un utilisateur est connecté à Gamelab,
- **Quand** il tape "Titre de Jeu inexistant" dans la barre de recherche,
- **Alors** un message s’affiche : "Aucun jeu correspondant à votre recherche."

Scénario 3 : Utilisateur non connecté

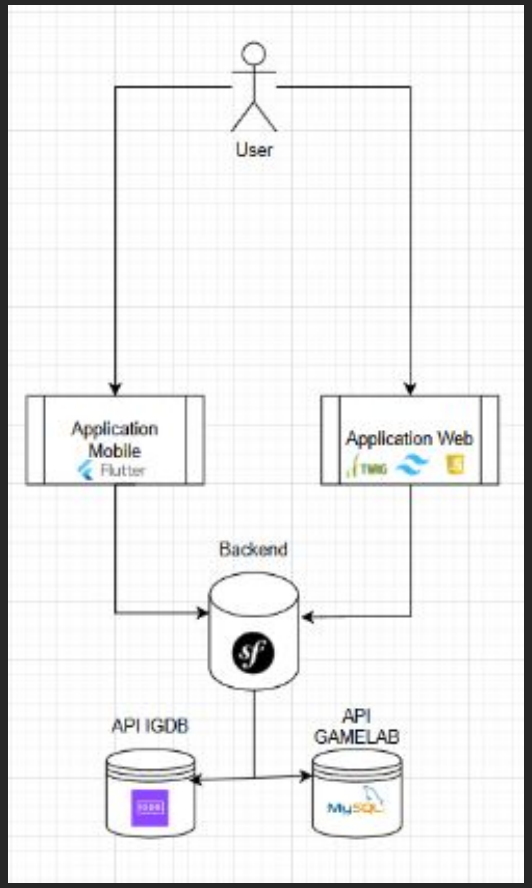
- **Étant donné** qu'un utilisateur n'est pas connecté à Gamelab,
- **Quand** il consulte une page de jeu (par exemple, "The Legend of Zelda: Breath of the Wild"),
- **Alors** il peut voir les détails du jeu.
- **Quand** il essaie d'ajouter le jeu à sa collection,
- **Alors** un message d'erreur s’affiche : "Erreur : veuillez vous connecter."

Ce user story est viable aussi pour les collections déjà existantes comme les collections “joué , favoris, wishlist et complété”.

8.Schéma MCD


Wishlist_Games
id (primary)
user_id (foreign key)
game_id
game_name
added_at
Backlogged_Games
id (primary)
user_id (foreign key)
game_id
game_name
added_at
Collection_Game
id (primary)
collection_id (foreign key)
game_id
game_name
updated_at
0,N
0,N
0,N
0,N
0,1
0,N
0,N
Favorite_Games
id (primary)
user_id (foreign key)
game_id
game_name
added_at
User
id (primary)
email (unique)
password
username
Completed_Games
id (primary)
user_id (foreign key)
game_id
game_name
added_at
Custom_Collection
id (primary)
user_id (foreign key)
name
created_at
updated_at
0,N
0,N
Abandonned_Games
id (primary)
user_id (foreign key)
game_id
game_name
added_at
Played_Games
id (primary)
user_id (foreign key)
game_id
game_name
added_at

9.Schéma d'architecture



docker

Conteneurisation du back-end avec Docker

10.Organisation

https://trello.com/b/jfcl9QHK/gamelab

Rédaction du dossier du projet en continue pendant le développement

Trello

git

11. Les API



Gamelab

11. Les API



+300 000 Jeux

REST

OAuth 2.0

exemple d'utilisation des endpoints de l'api

```php
/**
 * Récupère les détails d'un jeu à partir de son ID.
 *
 * @param int $gameId
 * @return Game
 * @throws \Exception
 */
public function fetchGameDetails(int $gameId): Game
{
    $query = <<<EOT
    fields id,name,summary,cover.url,screenshots.image_id,videos.video_id,genres.name,platforms.name,first_release_date;
    where id = {$gameId};
    EOT;
    $data = $this->makeRequest('/games', $query);

    if (empty($data)) {
        throw new \Exception('Jeu non trouvé');
    }

    return Game::fromArray($data[0]);
}
```

11. Les API

Gamelab

Datas de l’application

REST

JWT

Exemple API Entité

```
#[ApiResource(
    normalizationContext: ['groups' => ['user:read']],
    denormalizationContext: ['groups' => ['user:write']],
    operations: [
        new GetCollection(security: "is_granted('ROLE_ADMIN')"), // liste des utilisateurs (admin seulement)
        new GetCollection(
            uriTemplate: '/public/users',
            normalizationContext: ['groups' => ['user:public']],
            security: "is_granted('PUBLIC_ACCESS')"
        ), // liste publique des utilisateurs
        new Get(
            uriTemplate: '/public/users/{id}',
            normalizationContext: ['groups' => ['user:public']],
            security: "is_granted('PUBLIC_ACCESS')"
        ), // profil public d'un utilisateur
        new Post(),          // création d'un utilisateur
        new Get( // détails d'un utilisateur
            security: "object == user or is_granted('ROLE_ADMIN')",
            securityMessage: "Vous ne pouvez consulter que vos propres informations ou être administrateur."
        ),
        new Put( // modification d'un utilisateur
            security: "object == user or is_granted('ROLE_ADMIN')",
            securityMessage: "Vous ne pouvez modifier que vos propres informations ou être administrateur."
        ),
        new Delete( // suppression d'un utilisateur
            security: "object == user or is_granted('ROLE_ADMIN')",
            securityMessage: "Vous ne pouvez supprimer que votre propre compte ou être administrateur."
        )
    ]
)]
```

11. Les API





SWAGGER

Gamelab



Datas de l'application



REST



JWT

Récupérer les utilisateurs si nous sommes administrateur

```php
#[Route('/users', name: 'api_users_list', methods: ['GET'])]
#[IsGranted('ROLE_ADMIN')]
public function list(): JsonResponse
{
    $users = $this->userRepository->findAll();

    return $this->json([
        'users' => array_map(function($user) {
            return [
                'id' => $user->getId(),
                'email' => $user->getEmail(),
                'username' => $user->getUsername(),
                'roles' => $user->getRoles()
            ];
        }, $users)
    ]);
}
```

11. Les API







Datas de l'application

Gamelab



REST



JWT

```php
public function login(#[CurrentUser] ?User $user, JWTTokenManagerInterface $jwtManager)
{
    if (!$user) {
        return $this->json(['message' => 'Invalid credentials'], JsonResponse::HTTP_UNAUTHORIZED);
    }

    // Vérifier si l'email est vérifié, et refuser la connexion si non vérifié
    $isVerified = $user->isVerified();

    // Si l'utilisateur n'est pas vérifié, refuser la connexion
    if (!$isVerified) {
        return $this->json([
            'message' => 'Votre compte n\'est pas vérifié. Veuillez vérifier votre adresse email pour activer votre compte.',
            'verified' => false,
            'user_email' => $user->getEmail()
        ], JsonResponse::HTTP_FORBIDDEN);
    }

    // Générer un token JWT pour l'utilisateur
    $token = $jwtManager->create($user);

    // Retourner le token et les informations de l'utilisateur
    return $this->json([
        'token' => $token,
        'user' => [
            'id' => $user->getId(),
            'email' => $user->getEmail(),
            'roles' => $user->getRoles(),
            'username' => $user->getUsername(),
            'verified' => true
        ],
        'verified' => true,
        'user_email' => $user->getEmail()
    ], Response::HTTP_OK);
}
```

Le token est créé lors de la connexion

12.Test Unitaire




```php
public function testRegistrationAndLogin(): void
{
    try {
        // Étape 1: Inscription
        $this->client->request('POST', '/api/register', [], [], [
            'CONTENT_TYPE' => 'application/json'
        ], json_encode([
            'email' => $this->testEmail,
            'plainPassword' => 'StrongPassword123!@#',
            'name' => 'User Test'
        ]));

        $registerResponse = $this->client->getResponse();
        $registerStatusCode = $registerResponse->getStatusCode();
        $registerData = json_decode($registerResponse->getContent(), true);
        $userAlreadyExists = $registerStatusCode === Response::HTTP_BAD_REQUEST &&
                             isset($registerData['errors']) &&
                             is_array($registerData['errors']) &&
                             count(array_filter($registerData['errors'], function($error) {
                                 return strpos($error, 'déjà utilisé') !== false ||
                                        strpos($error, 'already used') !== false;
                             })) > 0;

        if ($registerStatusCode !== Response::HTTP_CREATED && !$userAlreadyExists) {
            $this->assertEquals(Response::HTTP_CREATED, $registerStatusCode,
                "L'inscription a échoué (code attendu: 201, reçu: $registerStatusCode)");
        }

        // Étape 2: Connexion avec les identifiants créés
        $this->client->request('POST', '/api/login', [], [], [
            'CONTENT_TYPE' => 'application/json'
        ], json_encode([
            'email' => $this->testEmail,
            'password' => 'StrongPassword123!@#'
        ]));

        $response = $this->client->getResponse();
        $statusCode = $response->getStatusCode();
        $responseData = json_decode($response->getContent(), true);

        $this->assertEquals(Response::HTTP_OK, $statusCode,
            "La connexion a échoué après inscription (code attendu: 200, reçu: $statusCode). " .
            "Réponse: " . $response->getContent());
        $this->assertArrayHasKey('email', $responseData, "La réponse ne contient pas le champ 'email'");
        $this->assertEquals($this->testEmail, $responseData['email']);
    } catch (\Exception $e) {
        $this->markTestIncomplete('Exception lors du test d\'inscription et connexion: ' . $e->getMessage());
    }
}
```

13. Docker de l'api Gamelab

```yaml
version: '3.8'

services:
  api:
    build:
      context: .
      dockerfile: Dockerfile.api
    container_name: gamelab_api
    ports:
      - "8000:8000"
    environment:
      DATABASE_URL: "mysql://root:password@database:3306/gamelab?serverVersion=mariadb-10.4.28"
      APP_ENV: "dev"
      APP_DEBUG: "1"
      MAILER_DSN: "smtp://mailhog:1025"
      MAILER_FROM_EMAIL: "noreply@gamelab.com"
      MAILER_FROM_NAME: "GameLab"
    depends_on:
      - database
      - mailhog

  database:
    image: mariadb:10.4.28
    container_name: gamelab_db
    environment:
      MYSQL_ROOT_PASSWORD: password
      MYSQL_DATABASE: gamelab
      MYSQL_USER: root
      MYSQL_PASSWORD: password
    volumes:
      - database_data:/var/lib/mysql:rw

  phpmyadmin:
    image: phpmyadmin/phpmyadmin
    container_name: gamelab_phpmyadmin
    depends_on:
      - database
    ports:
      - "8080:80"
    environment:
      PMA_HOST: database
      PMA_USER: root
      PMA_PASSWORD: password

  mailhog:
    image: mailhog/mailhog
    container_name: gamelab_mailhog
    ports:
      - "1025:1025" # SMTP server
      - "8025:8025" # Web UI
    environment:
      MH_STORAGE: maildir
      MH_MAILDIR_PATH: /tmp
    volumes:
      - mailhog_data:/tmp

volumes:
  database_data:
  mailhog_data:
```

13. Docker de l'api Gamelab

DockerFile.api qui sert à construire une image Docker prête à faire tourner mon API PHP

```
FROM php:8.2-cli

ARG DEBIAN_FRONTEND=noninteractive

WORKDIR /srv/app

COPY . .

RUN apt-get update \
 && apt-get install -y unzip git libzip-dev curl \
 && docker-php-ext-install pdo pdo_mysql zip \
 && rm -rf /var/lib/apt/lists/*

RUN curl -sS https://getcomposer.org/installer -o composer-setup.php \
 && php composer-setup.php --install-dir=/usr/local/bin --filename=composer \
 && rm composer-setup.php

RUN composer install --no-interaction --prefer-dist --optimize-autoloader

EXPOSE 8000

CMD ["php", "-S", "0.0.0.0:8000", "-t", "public"]
```

13. Landing Page

Le site est fait avec symfony et tailwind, il utilise également l'api IGDB pour le front.



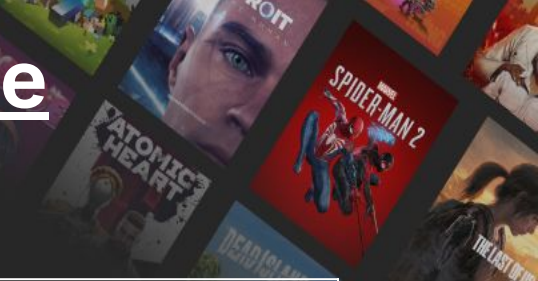
14. L’application

Gamelab





15. Veille Technologique



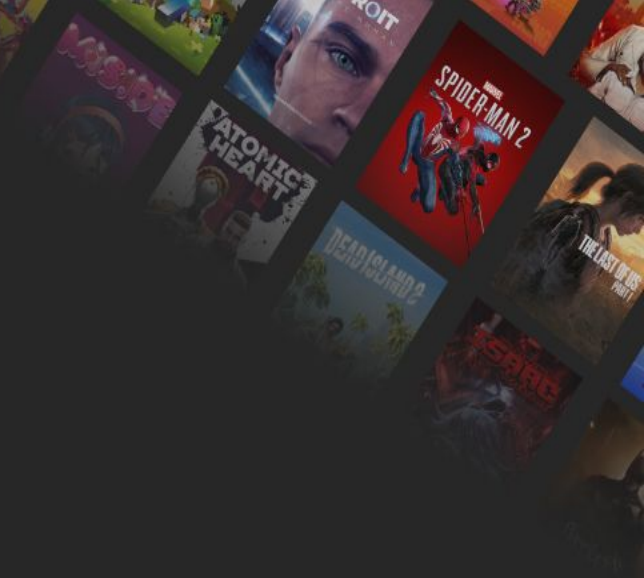
Empêcher des bots d'accéder à mon application	Utilisation de ReCaptcha V3 de google
Empêcher de se créer un compte via un faux mail	Utilisation d'une vérification d'email (VerifiedUserChecker) , grâce à un envoi de mail de confirmation via un serveur SMTP (MailHog)
Ne pas avoir à tout le temps envoyer une requête de connexion	Utilisation de JWT qui permet d'avoir un token et une utilisation Stateless
Sécurité du mot de passe	Utilisation d'une REGEX
Protections contre Injections SQL	Utilisation de requêtes paramétrées (Doctrine QueryBuilder + setParameter())

16. Améliorations futures



Mode Hors Ligne	Qui permettrait de pouvoir utiliser l'application de n'importe où même sans connexion internet
Multilingue	Car actuellement disponible uniquement en anglais
Pouvoir voir le profil des gens	Via une recherche et voir les jeux favoris, etc…
Intégration API Twtich	Pour voir quels jeux sont streamés afin d'avoir un meilleur aperçu
Intégration de HowLongToBeat	Pour savoir combien de temps un jeu est finissable
Prix des jeux	Pouvoir voir les prix
Guide des joueurs	Proposer aux joueurs de réaliser différents guides

17. Conclusion



Merci de m’avoir écouté

Avez-vous des questions ?